

ahora después de revisar el código (análisisEDA13(fin).ipynb) que entregaste funcionará bien quiero hacer un segundo código para hacer un entrenamiento para predecir el tipo de cultivo usando Fine-tuning de geoSAM y checkpoint, para generar o mejorar máscaras, y después entrenar un CNN de clasificar tipo de cultivo, siguiendo/cumpliendo los siguientes puntos:

Código SAM+CNN

Los archivos de imágenes son imagen sentinel-2 (t,b,h,w), máscaras TARGET(c,h,w), y máscaras ParcellIDs (h,w) agrupadas según el archivo estructura.txt; además las imágenes S2_*.npy son multitemporal.

0.terminal

```
python -m venv .venv  
.venv\Scripts\activate
```

Instalación en terminal para usar GPU RTX 4060, y las librerías necesarias para el código.

1.configuración inicial:

librerías, direcciones y variables comunes como ,la semilla (semilla=42)

2.limpieza de fechas de nubes:

Eliminar o enmascarar fechas con alta cobertura nubosa.

Calcular el brillo medio de las bandas visibles (banda 0:B2-azul, 1:B3-verde, 2:B4-rojo).

Calcular índice de nubosidad (NRGR): $NDGR = (B3 - B4) / (B3 + B4)$, este índice tiende a ser **positivo y alto** en zonas cubiertas por nubes, debido a la fuerte reflexión en el verde y el rojo.

Definición de umbral de nubosidad $NDGR > 0.2$ o brillo medio > 0.25 para identificar fechas con alta presencia de nubes.

Filtrado temporal para descartar automáticamente las fechas que superan estos umbrales, manteniendo únicamente las observaciones con condiciones atmosféricas despejadas.

3.calcular índices espectrales:

Agregar canales derivados para mejorar la información del modelo:

NDVI, GNDVI (vegetación), NDWI (agua), NDBI (suelo o urbano), SAVI (vigor vegetal), MSI (estrés hídrico)

4.Normalización y escalado

Escalar cada banda o índice entre 0 y 1 (o normalizar con media y desviación).

Esto estabiliza el aprendizaje y evita que bandas con valores altos dominen.

5. división de dataset (split 70/15/15)

División de split Entrenamiento (70%) / Validación (15%) / Prueba (15%).

Crear muestra aleatorias de los 2 entrenamientos train/val/test, el primero un split que tenga 100 IDs o 500 IDs dependiendo de lo que considere mejor, y el segundo un split que tenga todas las IDs, una de estas se usarán para los puntos 6.Fine-tuning SAM y después usar el mismo para el punto 7.Entrenamiento CNN clasificación de cultivos.

El split predeterminado es el primero que usa 100 IDs.

6.Fine-tuning SAM segmentación de imagen

tomar uno de los 3 split y usarlo en sam (usar split predeterminado)

clona repo de geosam <https://github.com/rafiibnsultan/GeoSAM>

Descargar y usar checkpoint sam_vit_h_4b8939.pth .

Convertir el archivo metadata.geojson a formato coco para su uso.

archivos necesarios: S2_*.npz y ParcelIDs_*.npz

necesita imagenes RGB o multibanda y máscara target(segmentaciones reales).

fine-tuning de SAM usará:**Input** de imágenes limpias y **Target** de máscaras reales (por cultivo o parcela); El resultado serán **máscaras predichas** (segmentaciones automáticas), que luego se usarán con la CNN.

SAM entrega una máscara generada mask_sam

7.Entrenamiento CNN clasificación de cultivos

usar el mismo split que se usó para SAM (usar split predeterminado).

usa tqdm para visualizar el progreso del entrenamiento.

archivos necesarios: S2_*.npz, TARGET_*.npz y máscara SAM refinada (mask_sam)

La CNN se entrena como modelo de segmentación semántica multiclase, para predecir el **tipo de cultivo por píxel**.

Al final la CNN entrega un mapa de clasificación agrícola donde cada píxel pertenece a una clase (por ejemplo, trigo, maíz, pradera, bosque, etc.).

opción de modelos: U-Net + ResNet34 (encoder)

hiperparametros:

batch size:15

Learning rate:1e-4 (encoder preentrenado) ReduceLROnPlateau o CosineAnnealing.

Epochs: 50

Usar mixed precision, optimizador, EarlyStopping(True patience=10) yAugmentation para rotación, flip entre otros.

el modelo entrenado es guardado usando torch.save(model.state_dict(), "best_model.pth"), nombre de archivo es correspondiente al split usado, el primer split "best_model100.pth", el segundo es "best_model500.pth", y el tercero es "best_modelT.pth".

CNN entrega un mapa de clases agrícolas TARGET_pred, esto es una predicción del tipo de cultivo por píxel

8. evaluar y guardar modelo

métricas para evaluar IoU, DICE, Accuracy, F1-score

8.1. Evaluar sam

IoU (Intersection over Union), Dice Coefficient (F1 de segmentación), Precision, recall, Boundary F1 (BFScore)

8.2. Evaluar CNN

Accuracy (global), mIoU (mean Intersection over Union), Precision / Recall / F1 (por clase), Cohen's Kappa, Macro-F1 o Weighted-F1, matriz de confusión.

8.3. graficar métricas

graficar distribución de clases reales de tipo de cultivo.

Graficar las métricas para observar las diferencias de máscaras reales de las predichas.

Graficar las métricas para observar las diferencias entre datos reales, de datos predichos.

8.4. visualización de comparación

mostrar lista de IDs disponibles para poder elegir una de esas para su visualización comparativa.

Graficar imágenes de comparación de tipo de cultivo real, con tipo de cultivo predicho, para visualizar sus diferencias de las máscaras real de la entregada por la predicción.

error de código

ubicación: punto 2

ubicación de error: punto 3

revisar número de workers porque recuerdo que daba error por la paralelización, eliminar o cambiar num_workers a 0

ubicación: split

ubicación de error: punto 3

revisar código para hacer un directorio de las IDs (dataframe/CSV), para asegurarse de cada S2_*.npy/ID de archivo tenga "TARGET_*.npy" y "ParcelIDs_*.npy" respectivo.

opción: el dataframe solo usara las IDs que tiene esos 3 archivos antes de hacer el split

enviar informe con video de presentación al profe